

Grupo: Deivson Jefferson Gomes, Eduardo Santos de Oliveira, Isaac Lima
Yungo, Isaque Luz Sousa, João Vitor Silva Santos

Projeto Integrador: Preferência

São Paulo

2026

Grupo: Deivson Jefferson Gomes, Eduardo Santos de Oliveira, Isaac Lima Yungo, Isaque Luz Sousa, João Vitor Silva Santos

Projeto Integrador: Preferência

Projeto Integrador 1

Centro Universitário Senac
Análise e Desenvolvimento de Sistemas

Orientador: Orlando Clinio Patriarcha

São Paulo
2026

Resumo

Nós iremos criar Um jogo no estilo “Would You Rather” e "The higher or Lower Games" consiste em apresentar ao jogador duas opções diferentes, geralmente curiosas, difíceis ou divertidas, e pedir que ele escolha qual opção ele acha ter mais relevância e preferência para outras pessoas no mundo. O foco principal está na tomada de decisão e na reflexão sobre as situações propostas, muitas vezes gerando respostas inesperadas ou discussões interessantes. É uma versão simples, o jogo apenas mostra as opções e registra a escolha do jogador.

A implementação em Java pode ser feita de forma básica utilizando o console, com classes para representar as perguntas e lógica para controlar o fluxo do jogo, ou pode evoluir para interfaces gráficas mais interativas.

Palavras-chave: Tomada de decisão, Reflexão, Java.

Lista de Figuras

Figura 1 – Fluxograma do sistema	7
--	---

Sumário

1	INTRODUÇÃO	5
1.1	Objetivo	5
1.2	Finalidade	5
2	FLUXOGRAMA	7
3	EXPLICAÇÃO DO CÓDIGO	8
3.1	Implementação	8
3.1.1	Visão Geral e Arquitetura	8
3.1.2	Classe Perguntas	8
3.1.3	Método main() — ponto de entrada	9
3.1.4	Método menu()	10
3.1.5	Método jogo() e validações	11
3.1.6	Método jogarPartida() — loop principal	11
3.1.7	Métodos mostrarPergunta() e lerResposta()	12
3.1.8	Métodos acertou() e errou()	13
3.1.9	Método menuGameOver()	14
3.1.10	Método perguntas() — banco de dados	15
3.1.11	Métodos auxiliares e ConsoleColors	16
4	CÓDIGO FONTE	18
4.1	Versão Final	18
5	CONCLUSÃO	41
	REFERÊNCIAS BIBLIOGRÁFICAS	42

1 Introdução

1.1 Objetivo

O objetivo do projeto é desenvolver um jogo de perguntas e respostas (quiz) utilizando a linguagem Java, com foco em proporcionar uma experiência objetiva, simples e divertida para os usuários. O sistema deverá apresentar perguntas de forma interativa, permitindo que o jogador teste seus conhecimentos de maneira dinâmica e intuitiva. Além disso, o projeto busca aplicar conceitos de programação orientada a objetos, lógica de programação e interação com o usuário, tornando o aprendizado prático e envolvente.

Além do entretenimento, o projeto também possui um caráter acadêmico e prático, buscando aplicar conceitos fundamentais da programação em Java, como lógica de programação, estruturas de repetição, condicionais, classes e princípios da programação orientada a objetos. Dessa forma, o desenvolvimento do quiz servirá não apenas como um produto final funcional, mas também como uma forma de aprimorar habilidades técnicas na área de desenvolvimento de software.

O jogo contará com funcionalidades como sistema de pontuação, validação de respostas, níveis de dificuldade, categorias de perguntas e feedback ao usuário após cada rodada. A interface será planejada para ser intuitiva e agradável, garantindo que o usuário consiga navegar facilmente pelas opções disponíveis. Dependendo da evolução do projeto, poderão ser implementados recursos adicionais, como ranking de jogadores, cronômetro para respostas, banco de perguntas e armazenamento de resultados.

Outro ponto importante do projeto é estimular o raciocínio rápido, a aprendizagem e a competitividade saudável entre os usuários. O quiz poderá ser utilizado tanto para fins recreativos quanto educacionais, tornando-se uma ferramenta versátil para diferentes contextos, como estudos, treinamentos ou momentos de lazer.

Portanto, o projeto busca unir tecnologia, aprendizado e diversão em uma única aplicação, demonstrando como a programação em Java pode ser utilizada para criar soluções criativas, funcionais e atrativas.

1.2 Finalidade

A finalidade deste projeto é desenvolver um jogo de quiz em Java que proporcione entretenimento, aprendizado e interação aos usuários de maneira simples, dinâmica e acessível. O sistema tem como propósito estimular o raciocínio lógico, a agilidade mental e o conhecimento geral dos jogadores por meio de perguntas e respostas organizadas em

diferentes categorias e níveis de dificuldade.

Além do aspecto recreativo, o projeto possui finalidade educacional e acadêmica, permitindo a aplicação prática de conceitos de programação em Java, como programação orientada a objetos, estruturas de decisão, repetição, manipulação de dados e interação com o usuário. Dessa forma, o desenvolvimento do jogo contribui tanto para o aperfeiçoamento técnico dos desenvolvedores quanto para a criação de uma aplicação funcional e intuitiva.

O projeto também busca demonstrar como a tecnologia pode ser utilizada para unir diversão e aprendizado em uma única plataforma, incentivando a participação dos usuários de forma interativa e motivadora.

2 Fluxograma

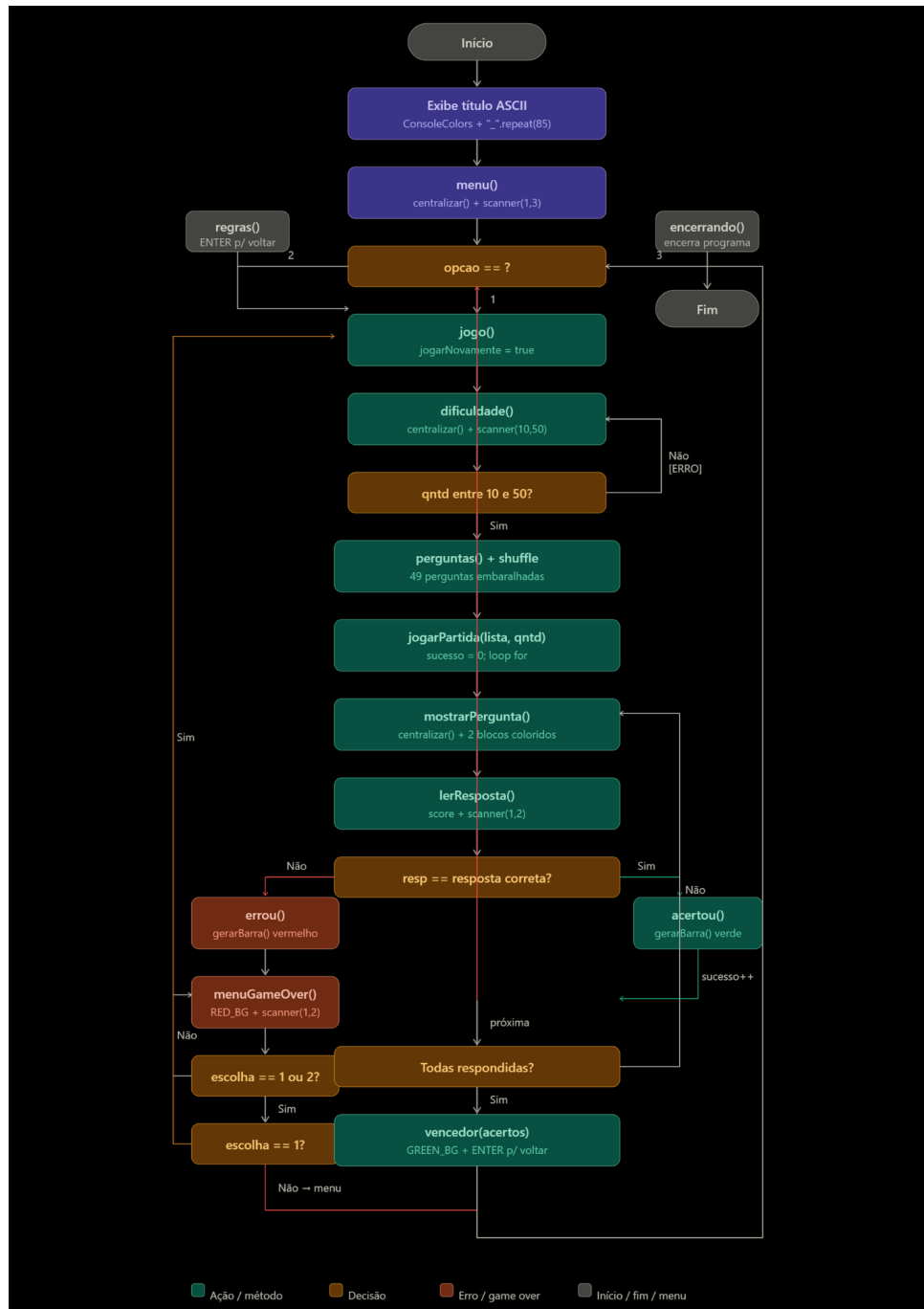


Figura 1 – Fluxograma do sistema

3 Explicação do Código

3.1 Implementação

O jogo Preferência foi desenvolvido em Java com execução via terminal, organizado dentro do pacote projeto. O programa é composto por duas classes: Perguntas, responsável por modelar os dados de cada questão, e ProjetoIntegrador, que concentra toda a lógica de execução, navegação e interação com o jogador. A dinâmica consiste em apresentar duas opções ao jogador, que deve adivinhar qual delas recebeu mais votos em uma pesquisa de popularidade.

3.1.1 Visão Geral e Arquitetura

O código é dividido em dois blocos principais. A classe Perguntas funciona como um modelo de dados simples, armazenando as informações de cada questão. A classe ProjetoIntegrador reúne o ponto de entrada do programa e todos os métodos estáticos responsáveis pelo fluxo do jogo. A navegação entre as telas é controlada por um loop while no método main(), que mantém o menu principal ativo enquanto o jogador não escolher a opção de saída. O objeto Scanner é declarado como atributo estático da classe, sendo compartilhado entre todos os métodos. A leitura de entradas numéricas é centralizada no método scanner(), que encapsula a validação e o tratamento de erros em um único lugar, eliminando repetição de código e tornando o programa mais robusto.

```
1 package projeto;
2 import java.util.*;
3
4 class Perguntas { ... }
5
6 public class ProjetoIntegrador {
7     static Scanner leia = new Scanner(System.in);
8     public static void main(String[] args) { ... }
9 }
```

3.1.2 Classe Perguntas

A classe Perguntas é responsável por representar cada questão do jogo. Ela possui cinco atributos do tipo String: pergunta1 e pergunta2 armazenam os textos das duas opções exibidas ao jogador; resultado1 e resultado2 guardam os percentuais de popularidade, revelados após a resposta; e resposta armazena "1" ou "2", indicando qual das opções é a

correta. O construtor recebe os cinco valores como parâmetros e os atribui diretamente aos atributos, sem nenhuma lógica adicional, funcionando apenas como um container de dados para cada pergunta do jogo.

```
1  class Perguntas {
2      String pergunta1;
3      String pergunta2;
4      String resposta;
5      String resultado1;
6      String resultado2;
7
8      Perguntas(String p1, String p2, String r1, String r2,
9          String re) {
10         pergunta1 = p1;
11         pergunta2 = p2;
12         resultado1 = r1;
13         resultado2 = r2;
14         resposta = re;
15     }
16 }
```

3.1.3 Método main() — ponto de entrada

O método main() é o ponto de entrada do programa e executa três responsabilidades principais. Primeiro, exibe o título do jogo em arte ASCII colorida no terminal, utilizando as constantes da classe ConsoleColors para aplicar as cores vermelho e ciano. O separador de linha é gerado dinamicamente com `"_".repeat(85)`, evitando strings fixas repetidas. Em seguida, inicializa a variável `opcao` e entra em um loop while que se mantém ativo enquanto o jogador não escolher a opção 3. Dentro do loop, o método `menu()` é chamado para exibir as opções e capturar a escolha do jogador, que é direcionada por um switch para um de três caminhos: iniciar o jogo com `jogo()`, exibir as regras com `regras()`, ou encerrar o programa com `encerrando()`. Entradas inválidas exibem uma mensagem de erro em vermelho com o prefixo [ERRO]. Ao sair do loop, o Scanner é fechado com `leia.close()`.

```
1  public static void main(String[] args) {
2
3      System.out.println(
4          ConsoleColors.RED_BOLD + "  _____  ..." +
5          ConsoleColors.RESET + "_".repeat(85) + "\n");
6
7      int opcao = 0;
8  }
```

```
9      while (opcao != 3) {
10          opcao = menu();
11
12          switch (opcao) {
13              case 1: jogo();          break;
14              case 2: regras();        break;
15              case 3: encerrando();    break;
16              default:
17                  System.out.println(ConsoleColors.RED + "[ERRO]
                  Valor Invalido");
18          }
19      }
20      leia.close();
21 }
```

3.1.4 Método menu()

O método menu() exibe o menu principal do jogo formatado com bordas de = e largura fixa de 85 caracteres, utilizando o método centralizar() para alinhar o conteúdo ao centro de cada linha. As três opções disponíveis — iniciar, ver as regras ou sair — são apresentadas entre colchetes no formato [1], [2] e [3]. A leitura da escolha do jogador é feita pelo método scanner(), que aceita apenas valores inteiros entre 1 e 3, exibindo mensagens de erro e solicitando nova entrada automaticamente em caso de valor inválido ou entrada não numérica. O valor inteiro escolhido é retornado para o switch no main().

```
1  public static int menu() {
2      int largura = 85;
3      int interna = largura - 2;
4
5      System.out.println("+" + "=".repeat(interna) + "+");
6      System.out.printf("|%s|\n", centralizar("MENU PRINCIPAL",
          interna));
7      System.out.println("+" + "=".repeat(interna) + "+");
8      System.out.printf("|%s|\n", centralizar("[1] Iniciar",
          interna));
9      System.out.printf("|%s|\n", centralizar("[2] Regras",
          interna));
10     System.out.printf("|%s|\n", centralizar("[3] Sair  ",
          interna));
11     System.out.println("+" + "=".repeat(interna) + "+");
12
13     System.out.print("Escolha uma opcao: ");
```

```
14     return scanner(leia, 1, 3);  
15 }
```

3.1.5 Método jogo() e validações

O método `jogo()` controla o ciclo completo de uma sessão de jogo por meio de um loop `while` com a variável `jogarNovamente`, que permite iniciar uma nova partida sem retornar ao menu principal. A cada iteração, o método chama `dificuldade()`, que exibe a tela de configuração de partida e já realiza a leitura e validação da quantidade de perguntas desejada internamente, retornando o valor inteiro diretamente. O método `jogo()` então verifica se o valor está entre 10 e 50 e, caso positivo, carrega a lista de perguntas com `perguntas()`, embaralha com `Collections.shuffle()` e inicia a partida com `jogarPartida()`. O retorno booleano desse método determina se o loop continua ou encerra.

```
1  public static void jogo() {  
2      boolean jogarNovamente = true;  
3  
4      while (jogarNovamente) {  
5          int qntd = dificuldade();  
6  
7          if (qntd >= 10 && qntd <= 50) {  
8              ArrayList<Perguntas> lista = perguntas();  
9              Collections.shuffle(lista);  
10             jogarNovamente = jogarPartida(lista, qntd);  
11         } else {  
12             System.out.println(ConsoleColors.RED + "[ERRO]  
13                 Valor Invalido");  
14         }  
15     }  
16 }
```

3.1.6 Método jogarPartida() — loop principal

O método `jogarPartida()` é o núcleo do jogo. Ele recebe a lista embaralhada de perguntas e a quantidade solicitada pelo jogador, percorrendo a lista com um loop `for` que respeita ambos os limites. A cada iteração, a pergunta atual é exibida por `mostrarPergunta()` e a resposta é lida por `lerResposta()`. Se a resposta estiver correta, `acertou()` é chamado e o contador sucesso é incrementado. Se o jogador errar, `errou()` é chamado e em seguida `menuGameOver()` captura a escolha. Caso o jogador insira um valor fora de 1 ou 2, um loop `do-while` exibe a mensagem de erro e reapresenta o menu até que uma opção válida seja fornecida. Escolher 1 retorna `true`, reiniciando a partida em `jogo()`; escolher 2 retorna

false, voltando ao menu principal. Se o jogador acertar todas as perguntas sem errar nenhuma, o método chama vencedor() com o total de acertos e retorna false.

```
1 public static boolean jogarPartida(ArrayList<Perguntas> lista,
2   int qntd) {
3
4   for (int i = 0; i < qntd && i < lista.size(); i++) {
5       Perguntas atual = lista.get(i);
6
7       mostrarPergunta(atual, sucesso);
8       String resp = lerResposta(sucesso);
9
10      if (resp.equals(atual.resposta)) {
11          acertou(atual);
12          sucesso++;
13      } else {
14          errou(atual);
15          int escolha = menuGameOver(sucesso);
16
17          if (escolha == 1 || escolha == 2) {
18              return escolha == 1;
19          } else {
20              do {
21                  System.out.println(ConsoleColors.RED + "[
22                      ERRO] Valor Invalido");
23                  escolha = menuGameOver(sucesso);
24              } while (escolha != 1 && escolha != 2);
25          }
26      }
27      vencedor(sucesso);
28      return false;
29 }
```

3.1.7 Métodos mostrarPergunta() e lerResposta()

O método mostrarPergunta() exibe as duas opções lado a lado no terminal, cada uma dentro de um bloco com bordas formatadas. A opção 1 é colorida em ciano e a opção 2 em vermelho negrito, utilizando as constantes de ConsoleColors. O conteúdo de cada bloco é centralizado com o método centralizar(), que calcula automaticamente os espaços necessários para alinhar o texto dentro de uma largura fixa de 48 caracteres internos. Entre

os dois blocos, um símbolo V S indica visualmente que o jogador deve escolher entre as opções. O método lerResposta() exibe o score atual do jogador antes de solicitar a entrada, utilizando System.out.printf. A leitura é feita pelo método scanner(), que aceita apenas os valores inteiros 1 ou 2 e trata automaticamente entradas inválidas, retornando o valor como String via String.valueOf().

```

1  public static void mostrarPergunta(Perguntas atual, int sucesso
    ) {
2      int largura = 50;
3      int interna = largura - 2;
4
5      System.out.println(ConsoleColors.CYAN + "+" + "-".repeat(
        interna)
6          + "+ " + ConsoleColors.RED_BOLD + "  " + "+ " + "-".repeat
            (interna) + "+");
7
8      System.out.printf(ConsoleColors.CYAN + "|%s| "
9          + ConsoleColors.RED_BOLD + "  |%s|\n",
10          centralizar(atual.pergunta1, interna),
11          centralizar(atual.pergunta2, interna));
12
13      System.out.println(ConsoleColors.CYAN + "+" + "-".repeat(
        interna)
14          + "+ " + ConsoleColors.RED_BOLD + "  " + "+ " + "-".repeat
            (interna) + "+");
15 }
16
17 public static String lerResposta(int acertos) {
18     System.out.printf("Score: %d\n", acertos);
19     System.out.print("Qual e mais popular?: ");
20     int resposta = scanner(leia, 1, 2);
21     return String.valueOf(resposta);
22 }

```

3.1.8 Métodos acertou() e errou()

Os métodos acertou() e errou() recebem o objeto da pergunta atual e exibem o resultado ao jogador após cada resposta. Ambos chamam o método gerarBarra() para converter os percentuais de popularidade em uma representação visual com barras de e -. A barra tem sempre 10 posições, onde cada posição representa 10A diferença entre os dois está na cor e na mensagem exibida: acertou() imprime o texto em verde usando ConsoleColors.GREEN, enquanto errou() utiliza ConsoleColors.RED. Em ambos os casos,

as barras das duas opções são exibidas, sendo a opção 1 em ciano e a opção 2 em vermelho negrito, permitindo ao jogador comparar visualmente os percentuais independentemente do resultado.

```
1 public static void acertou(Perguntas atual) {
2     System.out.println(ConsoleColors.GREEN + "Acertou!\n");
3     System.out.println(ConsoleColors.CYAN    + gerarBarra(atual
4         .resultado1));
5     System.out.println(ConsoleColors.RED_BOLD + gerarBarra(
6         atual.resultado2));
7     System.out.println(ConsoleColors.RESET);
8 }
9
10 public static void errou(Perguntas atual) {
11     System.out.println(ConsoleColors.RED + "Errou!\n");
12     System.out.println(ConsoleColors.CYAN    + gerarBarra(atual
13         .resultado1));
14     System.out.println(ConsoleColors.RED_BOLD + gerarBarra(
15         atual.resultado2));
16     System.out.println(ConsoleColors.RESET);
17 }
```

3.1.9 Método menuGameOver()

O método `menuGameOver()` é chamado quando o jogador erra uma pergunta. Ele recebe o score acumulado até aquele momento e exibe a tela de Game Over formatada com bordas de = e largura fixa de 85 caracteres. O título "GAME OVER" é destacado com fundo vermelho utilizando `ConsoleColors.RED_BACKGROUND`, e o score final é exibido dentro do bloco antes da borda inferior. As duas opções disponíveis — tentar novamente e voltar ao menu — são apresentadas no formato [1] e [2], centralizadas com o método `centralizar()`. A leitura da escolha é feita pelo método `scanner()`, que aceita apenas valores entre 1 e 2 e trata entradas inválidas automaticamente. O valor inteiro escolhido é retornado para `jogarPartida()`, que decide o próximo passo do programa.

```
1 public static int menuGameOver(int acertos) {
2     int largura = 85;
3     int interna = largura - 2;
4
5     System.out.printf("|%s|\n",
6         ConsoleColors.RED_BACKGROUND + centralizar("GAME OVER",
7             interna) + ConsoleColors.RESET);
8     System.out.println("+ " + "=" .repeat(interna) + "+");
```

```

8      System.out.printf("|%s|\n", centralizar("[1] Tentar
          Novamente", interna));
9      System.out.printf("|%s|\n", centralizar("[2] Menu Principal
          ", interna));
10     System.out.println("+" + "=" .repeat(interna) + "+");
11     System.out.printf("|%s|\n", centralizar("Score: " + acertos
          , interna));
12     System.out.println("+" + "=" .repeat(interna) + "+");
13
14     System.out.print("\nEscolha uma opcao: ");
15     return scanner(leia, 1, 2);
16 }

```

3.1.10 Método perguntas() — banco de dados

O método `perguntas()` é responsável por criar e retornar o banco de perguntas do jogo. Ele instancia um `ArrayList` do tipo `Perguntas` e adiciona 49 objetos, cada um representando uma comparação entre duas opções com seus respectivos percentuais de popularidade baseados em votações reais. Como os dados estão fixos no próprio código, o método é chamado a cada nova partida e a lista resultante é imediatamente embaralhada por `Collections.shuffle()` no método `jogo()`, garantindo que a ordem das perguntas seja diferente a cada vez que o jogador jogar, mesmo sem alterar o banco de dados. O conteúdo desse método é idêntico ao da versão anterior do código.

```

1  public static ArrayList<Perguntas> perguntas() {
2      ArrayList<Perguntas> lista = new ArrayList<>();
3
4      lista.add(new Perguntas(
5          "Pepsi", "Coca-Cola",
6          "Pepsi - 35.0%", "Coca-Cola - 65.0%", "2"));
7
8      lista.add(new Perguntas(
9          "Gato", "Cachorro",
10         "Gato - 36.8%", "Cachorro - 63.2%", "2"));
11
12         // ... 47 perguntas restantes
13
14         return lista;
15     }

```

3.1.11 Métodos auxiliares e ConsoleColors

O código conta com dois métodos de formatação utilizados em todo o programa. O método `centralizar()` recebe um texto e uma largura, calcula os espaços necessários à esquerda e à direita e retorna a string centralizada, sendo usado em todos os blocos de menu, regras, vencedor e game over. O método `gerarBarra()` recebe a string de resultado, extrai o valor percentual com `substring` e `lastIndexOf`, converte para `double` e gera uma barra visual de 10 posições com `para` os cheios e `-` para os vazios. O método `scanner()` centraliza toda a leitura de entradas numéricas do programa. Ele utiliza um loop `while(true)` com `try-catch` para capturar entradas não numéricas, valida se o valor está dentro do intervalo permitido e solicita nova entrada com mensagem de erro até que um valor válido seja fornecido. Isso elimina o uso direto de `nextInt()` em qualquer parte do código. Os métodos `regras()`, `vencedor()` e `encerrando()` são de exibição. `regras()` e `vencedor()` utilizam o mesmo padrão de formatação com bordas de `=` e `centralizar()`, sendo que `vencedor()` exibe o título "WINNER" com fundo verde via `ConsoleColors.GREENBACKGROUND` e recebe o total de acertos como parâmetro. Ambos aguardam o jogador pressionar ENTER com `leia.nextLine()` antes de retornar. `encerrando()` exibe apenas uma mensagem simples de encerramento. A classe `ConsoleColors` é uma inner class estática que armazena as constantes de escape ANSI para colorir o terminal. Em relação à versão anterior, foram adicionadas as constantes `GREEN`, `GREEN_BACKGROUND`, `RED` e `RED_BACKGROUND`, utilizadas nas telas de acerto, erro, vencedor e game over.

```
1 public static String centralizar(String texto, int largura) {
2     if (texto.length() >= largura) return texto.substring(0,
        largura);
3     int esquerda = (largura - texto.length()) / 2;
4     int direita = largura - texto.length() - esquerda;
5     return " ".repeat(esquerda) + texto + " ".repeat(direita);
6 }
7
8 public static String gerarBarra(String resultado) {
9     String porcentagemTexto = resultado
10         .substring(resultado.lastIndexOf("-") + 1, resultado.
            indexOf("%"))
11         .trim();
12     double porcentagem = Double.parseDouble(porcentagemTexto);
13     int quantidade = (int) porcentagem / 10;
14     String barra = "[";
15     for (int i = 0; i < quantidade; i++) barra += "#";
16     for (int i = quantidade; i < 10; i++) barra += "-";
17     return resultado + " - " + barra + "]";
18 }
```

```
19
20 public static int scanner(Scanner leia, int min, int max) {
21     while (true) {
22         try {
23             int valor = Integer.parseInt(leia.nextLine().trim()
24                                         );
25             if (valor < min || valor > max) {
26                 System.out.println(ConsoleColors.RED + "[ERRO]
27                                     Opcao invalida!");
28                 System.out.print("Tente novamente: ");
29                 continue;
30             }
31             return valor;
32         } catch (Exception e) {
33             System.out.println(ConsoleColors.RED + "[ERRO]
34                                 Digite apenas numeros!");
35             System.out.print("Tente novamente: ");
36         }
37     }
38 }
39
40 public static class ConsoleColors {
41     public static final String RESET           = "\033[0m";
42     public static final String GREEN           = "\033[0;32m";
43     public static final String GREEN_BACKGROUND = "\033[42m";
44     public static final String RED             = "\033[0;31m";
45     public static final String RED_BACKGROUND  = "\033[41m";
46     public static final String CYAN           = "\033[0;36m";
47     public static final String RED_BOLD       = "\033[1;31m";
48 }
```

4 Código Fonte

4.1 Versão Final

Listing 4.1 – Versão Final

```

1  package projeto;
2
3  import java.util.*;
4
5  class Perguntas {
6
7      String pergunta1;
8      String pergunta2;
9      String resposta;
10     String resultado1;
11     String resultado2;
12
13     Perguntas(String p1, String p2, String r1, String r2,
14               String re) {
15         pergunta1 = p1;
16         pergunta2 = p2;
17         resultado1 = r1;
18         resultado2 = r2;
19         resposta = re;
20     }
21 }
22
23 public class ProjetoIntegrador {
24
25     static Scanner leia = new Scanner(System.in);
26
27     public static void main(String[] args) {
28
29         // TITULO
30         System.out.println(
31             ConsoleColors.RED_BOLD + " -----
32                                     ---
33                                     " +
34             ConsoleColors.CYAN + "          \n"

```

```

31         + ConsoleColors.RED_BOLD + "|_  __ \\
           . ' ..]
                                     " +
           ConsoleColors.CYAN + "(_)          \n"
32     + ConsoleColors.RED_BOLD + " | |__ ) |_ .--.
       .---.  _| |_ .---. " + ConsoleColors.CYAN
       + "_ .---. .---.  _ .---. .---.  __ ,--.
         \n"
33     + ConsoleColors.RED_BOLD + " | ___/[ ' ' \\]/
       /__\\\\\\'-| |-'/ /__\\\\\\" + ConsoleColors.
       CYAN + "[ ' ' \\]/ /__\\\\\[ '.-. | / /' ' \\][
         | ' ' \\ : \n"
34     + ConsoleColors.RED_BOLD + " _| |_ | | | |
       \\__., | | | \\__., " + ConsoleColors.CYAN
       + "| | | | \\__., | | | | \\__., | | //
         | |, \n"
35     + ConsoleColors.RED_BOLD + "|____| [____]
       '.__.' [____] '.__.'" + ConsoleColors.CYAN +
       "[____] '.__.'[____| |__]' .__.'[____]\\\"'-;
       __/ \n"
36     + "

       \n"
37     + ConsoleColors.RESET + "_".repeat(85) + "\n");
38
39     int opcao = 0;
40
41     // DECISAO - MENU
42     while (opcao != 3) {
43
44         opcao = menu();
45
46         switch (opcao) {
47
48             case 1:
49                 jogo();
50                 break;
51
52             case 2:
53                 regras();
54                 break;
55

```

```
56         case 3:
57             encerrando();
58             break;
59
60         default:
61             System.out.println(ConsoleColors.RED + "[
62                 ERRO] Valor Invalido");
63     }
64 }
65 leia.close();
66 }
67
68 // RODAR O JOGO
69 public static void jogo() {
70
71     boolean jogarNovamente = true;
72
73     while (jogarNovamente) {
74
75         int qntd = dificuldade();
76
77         if (qntd >= 10 && qntd <= 50) {
78
79             ArrayList<Perguntas> lista = perguntas();
80
81             Collections.shuffle(lista);
82
83             jogarNovamente = jogarPartida(lista, qntd);
84
85         } else {
86             System.out.println(ConsoleColors.RED + "[ERRO]
87                 Valor Invalido");
88         }
89     }
90
91 // RODAR PARTIDA
92 public static boolean jogarPartida(ArrayList<Perguntas>
93     lista, int qntd) {
94
95     int sucesso = 0;
```

```
95
96     for (int i = 0; i < qntd && i < lista.size(); i++) {
97
98         Perguntas atual = lista.get(i);
99
100        mostrarPergunta(atual, sucesso);
101
102        String resp = lerResposta(sucesso);
103
104        System.out.println(ConsoleColors.RESET + "_".repeat
105                               (106));
106
107        if (resp.equals(atual.resposta)) {
108            acertou(atual);
109            sucesso++;
110        } else {
111            errou(atual);
112            int escolha = menuGameOver(sucesso);
113
114            if (escolha == 1 || escolha == 2) {
115                if (escolha == 1) {
116                    return true;
117                } else {
118                    return false;
119                }
120            } else {
121                do {
122                    System.out.println(ConsoleColors.RED +
123                                           "[ERRO] Valor Invalido");
124                    System.out.println(ConsoleColors.RESET)
125                        ;
126                    escolha = menuGameOver(sucesso);
127                } while (escolha != 1 && escolha != 2);
128            }
129        }
130        vencedor(sucesso);
131        return false;
132    }
133    // CARREGAR PERGUNTA
```

```
134     public static void mostrarPergunta(Perguntas atual, int
        sucesso) {
135
136         int largura = 50;
137         int interna = largura - 2;
138
139         System.out.println("_".repeat((largura * 2) + 6));
140
141         // Borda Superior
142         System.out.println(
143             ConsoleColors.CYAN + "+" + "-".repeat(interna)
                + "+ " + ConsoleColors.RED_BOLD + "  + " +
                "-".repeat(interna) + "+"
144         );
145
146         // Conteudo
147         System.out.printf(
148             ConsoleColors.CYAN + "|%s|  " + ConsoleColors.
                RED_BOLD + "  |%s|\n",
149             centralizar(atual.pergunta1, interna),
150             centralizar(atual.pergunta2, interna)
151         );
152         System.out.printf(
153             ConsoleColors.CYAN + "|%s|  V" + ConsoleColors.
                RED_BOLD + "  |%s|\n",
154             centralizar("", interna),
155             centralizar("", interna)
156         );
157         System.out.printf(
158             ConsoleColors.CYAN + "|%s|  " + ConsoleColors.
                RED_BOLD + "S  |%s|\n",
159             centralizar("ESCOLHA 1", interna),
160             centralizar("ESCOLHA 2", interna)
161         );
162         System.out.printf(
163             ConsoleColors.CYAN + "|%s|  " + ConsoleColors.
                RED_BOLD + "  |%s|\n",
164             centralizar("", interna),
165             centralizar("", interna)
166         );
167
168         // Borda Inferior
```

```
169         System.out.println(
170             ConsoleColors.CYAN + "+" + "-".repeat(interna)
171             + "+ " + ConsoleColors.RED_BOLD + " +" +
172             "-".repeat(interna) + "+"
173         );
174
175         System.out.println(
176             ConsoleColors.RESET + "_".repeat((largura * 2)
177             + 6)
178         );
179
180         System.out.println();
181     }
182
183     // LER RESPOSTA
184     public static String lerResposta(int acertos) {
185
186         System.out.printf("Score: %d\n", acertos);
187         System.out.print("Qual e mais popular?: ");
188
189         int resposta = scanner(leia, 1, 2);
190
191         return String.valueOf(resposta);
192     }
193
194     // CARREGAR ACERTO
195     public static void acertou(Perguntas atual) {
196
197         System.out.println();
198         System.out.println(ConsoleColors.GREEN + "Acertou!\n");
199
200         System.out.println(ConsoleColors.CYAN + gerarBarra(
201             atual.resultado1));
202         System.out.println(ConsoleColors.RED_BOLD + gerarBarra(
203             atual.resultado2));
204
205         System.out.println(ConsoleColors.RESET);
206     }
207
208     // CARREGAR ERRO
209     public static void errou(Perguntas atual) {
```



```
206         System.out.println();
207         System.out.println(ConsoleColors.RED + "Erro!\n");
208
209         System.out.println(ConsoleColors.CYAN + gerarBarra(
210             atual.resultado1));
211         System.out.println(ConsoleColors.RED_BOLD + gerarBarra(
212             atual.resultado2));
213     }
214
215     // CARREGAR MENU
216     public static int menu() {
217
218         int largura = 85;
219         int interna = largura - 2;
220
221         System.out.println();
222
223         // Borda Superior
224         System.out.println("+" + "=".repeat(interna) + "+");
225
226         // Conteúdo
227         System.out.printf("|%s|\n",
228             centralizar("MENU PRINCIPAL", interna));
229         System.out.println("+" + "=".repeat(interna) + "+");
230         System.out.printf("|%s|\n", centralizar("", interna));
231         System.out.printf("|%s|\n",
232             centralizar("[1] Iniciar", interna));
233         System.out.printf("|%s|\n",
234             centralizar("[2] Regras", interna));
235         System.out.printf("|%s|\n",
236             centralizar("[3] Sair ", interna));
237         System.out.printf("|%s|\n", centralizar("", interna));
238
239         // Borda Inferior
240         System.out.println("+" + "=".repeat(interna) + "+");
241
242         System.out.print("Escolha uma opcao: ");
243         return scanner(leia, 1, 3);
244     }
245
```

```
246     // Banco de Dados
247     public static ArrayList<Perguntas> perguntas() {
248
249         ArrayList<Perguntas> lista = new ArrayList<>();
250
251         lista.add(new Perguntas(
252             "Pepsi",
253             "Coca-Cola",
254             "Pepsi - 35.0%",
255             "Coca-Cola - 65.0%",
256             "2"
257         ));
258
259         lista.add(new Perguntas(
260             "Gato",
261             "Cachorro",
262             "Gato - 36.8%",
263             "Cachorro - 63.2%",
264             "2"
265         ));
266
267         lista.add(new Perguntas(
268             "Batman",
269             "Superman",
270             "Batman - 53.4%",
271             "Superman - 46.6%",
272             "1"
273         ));
274
275         lista.add(new Perguntas(
276             "Verao",
277             "Inverno",
278             "Verao - 67.8%",
279             "Inverno - 32.2%",
280             "1"
281         ));
282
283         lista.add(new Perguntas(
284             "YouTube",
285             "Facebook",
286             "YouTube - 81.9%",
287             "Facebook - 18.1%",
```

```
288         "1"
289     ));
290
291     lista.add(new Perguntas(
292         "Windows",
293         "MacOS",
294         "Windows - 79.3%",
295         "MacOS - 20.7%",
296         "1"
297     ));
298
299     lista.add(new Perguntas(
300         "Michael Jackson",
301         "Freddie Mercury",
302         "Michael Jackson - 59.0%",
303         "Freddie Mercury - 41.0%",
304         "1"
305     ));
306
307     lista.add(new Perguntas(
308         "Teleportar",
309         "Viajar no tempo",
310         "Teleportar - 50.3%",
311         "Viajar no tempo - 49.7%",
312         "1"
313     ));
314
315     lista.add(new Perguntas(
316         "Calor",
317         "Frio",
318         "Calor - 23.0%",
319         "Frio - 77.0%",
320         "2"
321     ));
322
323     lista.add(new Perguntas(
324         "Fumar",
325         "Beber",
326         "Fumar - 29.5%",
327         "Beber - 70.5%",
328         "2"
329     ));
```

```
330
331     lista.add(new Perguntas(
332         "Correr 26 kilometros",
333         "Nadar 5 kilometros",
334         "Correr 26 kilometros - 50.4%",
335         "Nadar 5 kilometros - 49.6%",
336         "1"
337     ));
338
339     lista.add(new Perguntas(
340         "Se queimar",
341         "Se afogar",
342         "Se queimar - 31.5%",
343         "Se afogar - 68.5%",
344         "2"
345     ));
346
347     lista.add(new Perguntas(
348         "Ser famoso enquanto vivo",
349         "Ir para os livros de historia para sempre",
350         "Ser famoso enquanto vivo - 52.5%",
351         "Ir para os livros de historia para sempre -
352         47.5%",
353         "1"
354     ));
355
356     lista.add(new Perguntas(
357         "Perder a mao",
358         "Perder o pe",
359         "Perder a mao - 30.7%",
360         "Perder o pe - 69.3%",
361         "2"
362     ));
363
364     lista.add(new Perguntas(
365         "Dor no estomago",
366         "Dor de cabeca",
367         "Dor no estomago - 48.0%",
368         "Dor de cabeca - 52.0%",
369         "2"
370     ));
```

```
371         lista.add(new Perguntas(  
372             "Ver como tudo começou",  
373             "Ver como tudo acaba",  
374             "Ver como tudo começou - 45.9%",  
375             "Ver como tudo acaba - 54.1%",  
376             "2"  
377         ));  
378  
379         lista.add(new Perguntas(  
380             "Ser cantor",  
381             "Ser ator",  
382             "Ser cantor - 31.5%",  
383             "Ser ator - 68.5%",  
384             "2"  
385         ));  
386  
387         lista.add(new Perguntas(  
388             "Fruta",  
389             "Vegetal",  
390             "Fruta - 87.9%",  
391             "Vegetal - 12.1%",  
392             "1"  
393         ));  
394  
395         lista.add(new Perguntas(  
396             "Ir primeiro",  
397             "Ir por ultimo",  
398             "Ir primeiro - 49.4%",  
399             "Ir por ultimo - 50.6%",  
400             "2"  
401         ));  
402  
403         lista.add(new Perguntas(  
404             "Passar um ano no mar",  
405             "Passar um ano no espaco",  
406             "Passar um ano no mar - 31.0%",  
407             "Passar um ano no espaco - 69.0%",  
408             "2"  
409         ));  
410  
411         lista.add(new Perguntas(  
412             "Dirigir bebado",
```

```
413         "Dirigir com sono",
414         "Dirigir bebado - 39.0%",
415         "Dirigir com sono - 60.1%",
416         "2"
417     ));
418
419     lista.add(new Perguntas(
420         "Viver uma vida esquecivel",
421         "Ir para historia por algo terrivel",
422         "Viver uma vida esquecivel - 68.3%",
423         "Ir para historia por algo terrivel - 31.7%",
424         "1"
425     ));
426
427     lista.add(new Perguntas(
428         "Ser invencivel",
429         "Ser invisivel",
430         "Ser invencivel - 55.5%",
431         "Ser invisivel - 44.5%",
432         "1"
433     ));
434
435     lista.add(new Perguntas(
436         "Salario bom e emprego horrivel",
437         "Salario horrivel e emprego bom",
438         "Salario bom e emprego horrivel - 55.6%",
439         "Salario horrivel e emprego bom - 44.4%",
440         "1"
441     ));
442
443     lista.add(new Perguntas(
444         "Nao saber ler",
445         "Nao saber escrever",
446         "Nao saber ler - 25.4%",
447         "Nao saber escrever - 74.6%",
448         "2"
449     ));
450
451     lista.add(new Perguntas(
452         "Deserto do Saara",
453         "Polo Norte",
454         "Deserto do Saara - 34.9%",
```

```
455         "Polo Norte - 65.1%",
456         "2"
457     ));
458
459     lista.add(new Perguntas(
460         "Nao sentir tristeza",
461         "Nao sentir raiva",
462         "Nao sentir tristeza - 53.1%",
463         "Nao sentir raiva - 46.9%",
464         "1"
465     ));
466
467     lista.add(new Perguntas(
468         "Ir para o pos-vida",
469         "Renascer",
470         "Ir para o pos-vida - 41.9%",
471         "Renascer - 58.1%",
472         "2"
473     ));
474
475     lista.add(new Perguntas(
476         "Perder o paladar",
477         "Perder o olfato",
478         "Perder o paladar - 23.9%",
479         "Perder o olfato - 76.1%",
480         "2"
481     ));
482
483     lista.add(new Perguntas(
484         "Inteligente entre burros",
485         "Burro entre inteligentes",
486         "Inteligente entre burros - 78.7%",
487         "Burro entre inteligentes - 21.3%",
488         "1"
489     ));
490
491     lista.add(new Perguntas(
492         "Ganhar o Oscar",
493         "Ganhar o Premio Nobel",
494         "Ganhar o Oscar - 34.0%",
495         "Ganhar o Premio Nobel - 66.0%",
496         "2"
```

```
497         ));
498
499         lista.add(new Perguntas(
500             "Voz baixa",
501             "Voz alta",
502             "Voz baixa - 71.1%",
503             "Voz alta - 28.9%",
504             "1"
505         ));
506
507         lista.add(new Perguntas(
508             "Grecia antiga",
509             "Egito antigo",
510             "Grecia antiga - 75.8%",
511             "Egito antigo - 24.2%",
512             "1"
513         ));
514
515         lista.add(new Perguntas(
516             "Pokemon",
517             "Superherois",
518             "Pokemon - 48.2%",
519             "Superherois - 51.8%",
520             "2"
521         ));
522
523         lista.add(new Perguntas(
524             "Skittles",
525             "M&M's",
526             "Skittles - 52.2%",
527             "M&M's - 47.8%",
528             "1"
529         ));
530
531         lista.add(new Perguntas(
532             "Elefante",
533             "Rato",
534             "Elefante - 64.5%",
535             "Rato - 35.5%",
536             "1"
537         ));
538
```



```
539         lista.add(new Perguntas(  
540             "Pintura",  
541             "Escultura",  
542             "Pintura - 33.0%",  
543             "Escultura - 67.0%",  
544             "2"  
545         ));  
546  
547         lista.add(new Perguntas(  
548             "Comer",  
549             "Beber",  
550             "Comer - 54.6%",  
551             "Beber - 45.4%",  
552             "1"  
553         ));  
554  
555         lista.add(new Perguntas(  
556             "Ser pego traindo",  
557             "Descobrir traicao",  
558             "Ser pego traindo - 31.1%",  
559             "Descobrir traicao - 68.9%",  
560             "2"  
561         ));  
562  
563         lista.add(new Perguntas(  
564             "Viver no passado",  
565             "Viver no futuro",  
566             "Viver no passado - 29.1%",  
567             "Viver no futuro - 70.9%",  
568             "2"  
569         ));  
570  
571         lista.add(new Perguntas(  
572             "Ter somente riqueza",  
573             "Ter somente fama",  
574             "Ter somente riqueza - 78.0%",  
575             "Ter somente fama - 22.0%",  
576             "1"  
577         ));  
578  
579         lista.add(new Perguntas(  
580             "Vampiro",
```

```
581         "Lobisomem",
582         "Vampiro - 48.8%",
583         "Lobisomem - 51.2%",
584         "2"
585     ));
586
587     lista.add(new Perguntas(
588         "Emo",
589         "Gótico",
590         "Emo - 44.6%",
591         "Gótico - 55.4%",
592         "2"
593     ));
594
595     lista.add(new Perguntas(
596         "Ser atraente e pobre",
597         "Ser feio e rico",
598         "Ser atraente e pobre - 56.6%",
599         "Ser feio e rico - 43.4%",
600         "1"
601     ));
602
603     lista.add(new Perguntas(
604         "Michael Jordan",
605         "Kobe Bryant",
606         "Michael Jordan - 74.2%",
607         "Kobe Bryant - 25.8%",
608         "1"
609     ));
610
611     lista.add(new Perguntas(
612         "Saber quando mentem",
613         "Mentir sem ser pego",
614         "Saber quando mentem - 57.3%",
615         "Mentir sem ser pego - 42.7%",
616         "1"
617     ));
618
619     lista.add(new Perguntas(
620         "Designer Grafico",
621         "Arquiteto",
622         "Designer Grafico - 56.4%",
```

```
623         "Arquiteto - 43.6%",
624         "1"
625     ));
626
627     lista.add(new Perguntas(
628         "Taylor Swift",
629         "Beyonce",
630         "Taylor Swift - 49.9%",
631         "Beyonce - 50.1%",
632         "2"
633     ));
634
635     lista.add(new Perguntas(
636         "Nao celebrar o Natal",
637         "Nao celebrar aniversario",
638         "Nao celebrar o Natal - 39.8%",
639         "Nao celebrar aniversario - 60.2%",
640         "2"
641     ));
642
643     lista.add(new Perguntas(
644         "Policia",
645         "Bombeiro",
646         "Policia - 64.4%",
647         "Bombeiro - 35.6%",
648         "1"
649     ));
650
651     return lista;
652 }
653
654 // FUNÇÕES DE FORMATAÇÃO
655
656 public static String centralizar(String texto, int largura)
657 {
658     // Se o texto estiver dentro dos limites, não corrigir
659     if (texto.length() >= largura) {
660         return texto.substring(0, largura);
661     }
662
663     // Correção do espaço
```

```
664         int esquerda = (largura - texto.length()) / 2;
665         int direita = largura - texto.length() - esquerda;
666
667         // Centralizar
668         return " ".repeat(esquerda)
669             + texto
670             + " ".repeat(direita);
671     }
672
673     public static String gerarBarra(String resultado) {
674
675         // Pega a parte antes do %
676         String porcentagemTexto = resultado
677             .substring(resultado.lastIndexOf("-") + 1,
678                 resultado.indexOf("%"))
679             .trim();
680
681         // Converte para número
682         double porcentagem = Double.parseDouble(
683             porcentagemTexto);
684
685         // Pega o primeiro dígito inteiro
686         int quantidade = (int) porcentagem / 10;
687
688         // Barra
689         String barra = "[";
690         for (int i = 0; i < quantidade; i++) {
691             barra += "#";
692         }
693
694         for (int i = quantidade; i < 10; i++) {
695             barra += "-";
696         }
697
698         barra += "]";
699
700         return resultado + " - " + barra;
701     }
702
703     // FUNCIONALIDADES
704
705     // CARREGAR REGRAS
```

```
704     public static String regras() {
705
706         int largura = 85;
707         int interna = largura - 2;
708
709         System.out.println();
710
711         // Borda Superior
712         System.out.println("+" + "=" .repeat(interna) + "+");
713
714         // Conteudo
715         System.out.printf("|%s|\n",
716             centralizar("COMO JOGAR", interna));
717         System.out.println("+" + "=" .repeat(interna) + "+");
718         System.out.printf("|%s|\n",
719             centralizar("", interna));
720         System.out.printf("|%s|\n",
721             centralizar("Entre duas opcoes", interna));
722         System.out.printf("|%s|\n",
723             centralizar("descubra qual recebeu", interna));
724         System.out.printf("|%s|\n",
725             centralizar("mais votos!", interna));
726         System.out.printf("|%s|\n",
727             centralizar("", interna));
728         System.out.printf("|%s|\n",
729             centralizar("", interna));
730
731         // Borda Inferior
732         System.out.println("+" + "=" .repeat(interna) + "+");
733
734         System.out.print("Pressione ENTER para voltar...");
735         leia.nextLine();
736
737         return "";
738     }
739
740     // CARREGAR WINNER
741     public static String vencedor(int acertos) {
742
743         int largura = 85;
744         int interna = largura - 2;
745
```

```
746         System.out.println();
747
748         // Borda Superior
749         System.out.println("+" + "=".repeat(interna) + "+");
750
751         // Conteudo
752         System.out.printf("|%s|\n",
753             ConsoleColors.GREEN_BACKGROUND + centralizar("
754             WINNER", interna) + ConsoleColors.RESET);
755         System.out.println("+" + "=".repeat(interna) + "+");
756         System.out.printf("|%s|\n",
757             centralizar("", interna));
758         System.out.printf("|%s|\n",
759             centralizar("Parabens!", interna));
760         System.out.printf("|%s|\n",
761             centralizar("Voce acertou todas as perguntas!",
762             interna));
763         System.out.printf("|%s|\n",
764             centralizar("Score: " + acertos, interna));
765         System.out.printf("|%s|\n",
766             centralizar("", interna));
767
768         // Borda Inferior
769         System.out.println("+" + "=".repeat(interna) + "+");
770
771         System.out.print("Pressione ENTER para voltar...");
772         leia.nextLine();
773
774         return "";
775     }
776
777     // CARREGAR GAME OVER
778     public static int menuGameOver(int acertos) {
779
780         int largura = 85;
781         int interna = largura - 2;
782
783         System.out.println();
784
785         // Borda Superior
786         System.out.println("+" + "=".repeat(interna) + "+");
```

```
786         // Conteudo
787         System.out.printf("%s\n",
788             ConsoleColors.RED_BACKGROUND + centralizar("
789             GAME OVER", interna) + ConsoleColors.RESET);
790         System.out.println("+" + "=".repeat(interna) + "+");
791         System.out.printf("%s\n",
792             centralizar("", interna));
793         System.out.printf("%s\n",
794             centralizar("[1] Tentar Novamente", interna));
795         System.out.printf("%s\n",
796             centralizar("[2] Menu Principal", interna));
797         System.out.printf("%s\n",
798             centralizar("", interna));
799         System.out.println("+" + "=".repeat(interna) + "+");
800         System.out.printf("%s\n",
801             centralizar("Score: " + acertados, interna));
802
803         // Borda Inferior
804         System.out.println("+" + "=".repeat(interna) + "+");
805
806         System.out.print("\nEscolha uma opcao: ");
807
808         int escolha = scanner(leia, 1, 2);
809
810         return escolha;
811     }
812
813     // CARREGAR QUANTIDADE DE PERGUNTAS
814     public static int dificuldade() {
815         int largura = 85;
816         int interna = largura - 2;
817
818         System.out.println();
819
820         // Borda Superior
821         System.out.println("+" + "=".repeat(interna) + "+");
822
823         // Conteudo
824         System.out.printf("%s\n",
825             centralizar("CONFIGURACAO DE PARTIDA", interna)
826             );
827         System.out.println("+" + "=".repeat(interna) + "+");
```

```
826         System.out.printf("|%s|\n",
827                             centralizar("", interna));
828         System.out.printf("|%s|\n",
829                             centralizar("Minimo: 10 | Maximo: 50", interna)
830                                     );
831         System.out.printf("|%s|\n",
832                             centralizar("", interna));
833
834         // Borda Inferior
835         System.out.println("+" + "=".repeat(interna) + "+");
836
837         System.out.print("\nQuantas perguntas?: ");
838
839         int qntd = scanner(leia, 10, 50);
840
841         return qntd;
842     }
843
844     // ENCERRAR O PROGRAMA
845     public static void encerrando() {
846         System.out.println("Encerrando...");
847     }
848
849     // Verificação de erros
850     public static int scanner(Scanner leia, int min, int max) {
851         while (true) {
852             try {
853                 String entrada = leia.nextLine().trim();
854                 int valor = Integer.parseInt(entrada);
855
856                 if (valor < min || valor > max) {
857                     System.out.println(
858                         ConsoleColors.RED
859                         + "[ERRO] Opcao invalida!"
860                         + ConsoleColors.RESET
861                     );
862
863                     System.out.print("Tente novamente: ");
864                     continue;
865                 }
866                 return valor;
```



```
867
868         } catch (Exception e) {
869             System.out.println(
870                 ConsoleColors.RED
871                 + "[ERRO] Digite apenas numeros!"
872                 + ConsoleColors.RESET
873             );
874             System.out.print("Tente novamente: ");
875         }
876     }
877 }
878
879 // Cores
880 public static class ConsoleColors {
881
882     public static final String RESET = "\033[0m";
883
884     public static final String GREEN = "\033[0;32m";
885     public static final String GREEN_BACKGROUND = "\033[42m";
886     ";
887     public static final String RED = "\033[0;31m";
888     public static final String RED_BACKGROUND = "\033[41m";
889     ;
890
891     public static final String CYAN = "\033[0;36m";
892     public static final String RED_BOLD = "\033[1;31m";
893 }
}
```

5 Conclusão

O desenvolvimento deste projeto permitiu aplicar, na prática, diversos conceitos fundamentais da programação Java do primeiro semestre. Durante a construção do jogo, foram utilizados recursos como classes, métodos, estruturas de repetição, condicionais, listas dinâmicas com ArrayList entre outros.

O sistema foi desenvolvido com o objetivo de proporcionar uma experiência interativa ao usuário através de perguntas, usando como inspiração o jogo “Would You Rather”, onde o jogador deve adivinhar qual opção recebeu mais votos. Além do aspecto recreativo, o projeto contribuiu significativamente para o aprimoramento do raciocínio lógico, organização de código e estruturação de funcionalidades em um programa completo.

Outro ponto importante foi a implementação de recursos que melhoram a experiência do usuário, como menus interativos, sistema de pontuação, validação de respostas, reinício de partida e padronização de cores na String, com a visualização diretamente no terminal, utilizando cores ANSI (padrão universal de computação, onde o próprio terminal do sistema operacional entende que aquela parte da String deve ser colorida). Essas funcionalidades demonstram a preocupação com usabilidade e organização da aplicação.

Outro aspecto relevante foi a preocupação com a modularização do código, separando responsabilidades em métodos específicos, o que facilita a manutenção, compreensão e futuras atualizações da aplicação. Essa prática é essencial no desenvolvimento de sistemas maiores e contribui diretamente para a qualidade do software.

Por fim, o projeto atingiu seus objetivos principais, demonstrando na prática como conceitos aprendidos em sala podem ser aplicados no desenvolvimento de um software funcional. Além disso, abriu oportunidades para futuras melhorias, como implementação de banco de dados, interface gráfica, sistema de ranking e expansão do banco de perguntas, tornando o jogo ainda mais completo e dinâmico.

Referências Bibliográficas

Jogo WEB 1º de referência

<<https://www.higherlowergame.com/>>

Jogo WEB de 2º de referência

<<https://wouldyourather.app/pt/s/05tfw>>

Site com o code de cor no console (String)

<<https://goo.su/8xsI3>>

Titulo modificado no console

<<https://patorjk.com/software/taag/#p=display&f=Graffiti&t=Type+Something+&x=none&v=4&h=4&w=80&we=false>>

Site com as perguntas elaboradas e seu devido valores de respostas (porcetagem calculada a parte)

<<https://www.kaggle.com/datasets/charlieray668/would-you-rather>>